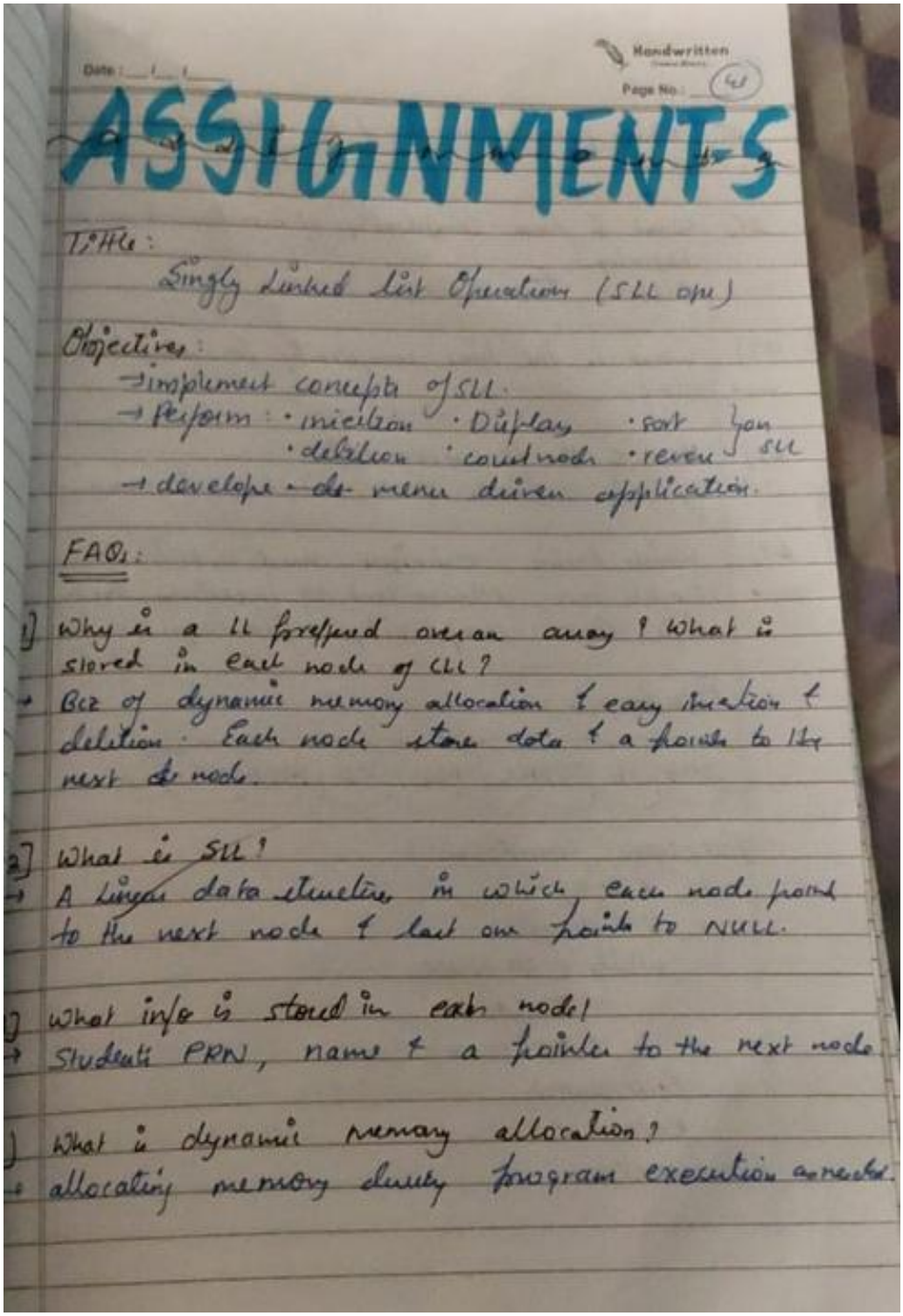
ASSIGNMENT – 05

Checked Assignment :



ASSIGNMENT-5

Title:
Singly Linked List Operations (SLL ops)

Objectives:
→ Implement concepts of SLL.
→ Perform: • insertion • Display • sort from SLL
• deletion • count node • reverse
→ develope a menu driven application.

FAQs:

1] Why is a LL preferred over an array? What is stored in each node of LL?
→ Bcz of dynamic memory allocation & easy insertion & deletion. Each node store data & a pointer to the next node.

2] What is SLL?
→ A linear data structure in which each node points to the next node & last one points to NULL.

3] What info is stored in each node?
→ Student PRN, name & a pointer to the next node.

4] What is dynamic memory allocation?
→ allocating memory during program execution as needed.

Q5) Which function is used to release allocated memory?
→ free()

Q6) What is time complexity of inserting a node at beginning?
→ O(1) [no traversal]

Q7) What is the time complexity for searching a node?
→ O(n)

Q8) How is the LL traversed?
→ By using 3 pointers prev, current & next

Q9) Which sorting technique used in LL?
→ Bubble Sort, Merge Sort or Insertion Sort with time complexity O(n log n).

Pseudocode:

COMPUTE TOTAL NO. OF MEMBERS

```
FUNCTION countMem()
    c ← 0
    t ← head
    while t ≠ NULL
        c ← c+1
        t ← t.next
    END WHILE
    Return c
END FUNCTION
```

Date / / Handwritten Page No. 41

DISPLAY ALL MEMBERS

```
FUNCTION DISP()
        t ← head
        IF head ≠ NULL
            print "CLUB is empty"
            Return
        END IF
        WHILE t ≠ NULL
            print t.PRN , t.N
            t ← t.NEXT
        END WHILE
END FUNCTION
```

SORTING A LL

```
FUNCTION SORT(head)
    FOR each node i in the LIST
        For each node j after i
            IF i.PRN > j.PRN
                SWAP i.PRN & j.PRN
                SWAP i.N & j.N
            End IF
        ENDFOR
    ENDFOR
    RETURN head
END FUNCTION
```

Date: __/__/____

MERGING 2 LL

```
FUNCTION MERGE (h1, h2)
    Create a dummy node
    tail ← dummy
    WHILE h1 ≠ Null ≠ h2 ≠ Null
        IF h1.PRN ≤ h2.PRN
            tail.NEXT ← h1
            h1 ← h1.NEXT
        ELSE
            tail.NEXT ← h2
            h2 ← h2.NEXT
        ENDIF
        tail ← tail.NEXT
    ENDWHILE
    IF h1 ≠ NULL
        tail.NEXT ← h1
    ELSE
        tail.NEXT ← h2
    END IF
    RETURN dummy.NEXT
END FUNCTION
```

Time complexity:

displaying : O(n)
counting : O(n)
reversing : O(n)
sorting : O(n²)

overall = O(n²)

Code :

```
#
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

struct Node {
    int prn;
    char name[50];
    struct Node *next;
};

struct Node *head = NULL;

struct Node* createNode(int prn, char name[]) {
    struct Node *newNode =
        (struct Node*)malloc(sizeof(struct Node));

    newNode->prn = prn;
    strcpy(newNode->name, name);
    newNode->next = NULL;
```

```
    return newNode;
}

// Add president
void addPresident(int prn, char name[]) {
    struct Node *newNode = createNode(prn, name);
    newNode->next = head;
    head = newNode;
}

// Add secretary
void addSecretary(int prn, char name[]) {
    struct Node *newNode = createNode(prn, name);

    if (head == NULL) {
        head = newNode;
        return;
    }

    struct Node *temp = head;

    while (temp->next != NULL)
```

```
        temp = temp->next;

    temp->next = newNode;
}

// Add member before secretary
void addMember(int prn, char name[]) {
    struct Node *newNode = createNode(prn, name);

    if (head == NULL || head->next == NULL) {
        printf("Add president and secretary first.\n");
        free(newNode);
        return;
    }

    struct Node *temp = head;

    while (temp->next->next != NULL)
        temp = temp->next;

    newNode->next = temp->next;
    temp->next = newNode;
```

```
}

// Delete president
void deletePresident() {
    if (head == NULL) {
        printf("Club is empty.\n");
        return;
    }

    struct Node *temp = head;
    head = head->next;
    free(temp);
}

// Delete secretary
void deleteSecretary() {
    if (head == NULL) {
        printf("Club is empty.\n");
        return;
    }

    if (head->next == NULL) {
```

```
        free(head);
        head = NULL;
        return;
    }

    struct Node *temp = head;

    while (temp->next->next != NULL)
        temp = temp->next;

    free(temp->next);
    temp->next = NULL;
}

// Delete member using PRN
void deleteMember(int prn) {
    struct Node *temp = head, *prev = NULL;

    while (temp != NULL && temp->prn != prn) {
        prev = temp;
        temp = temp->next;
    }
```

```
    if (temp == NULL) {
        printf("Member not found.\n");
        return;
    }

    if (prev == NULL)
        head = temp->next;
    else
        prev->next = temp->next;

    free(temp);
    printf("Member deleted.\n");
}

// Count members
int countMembers() {
    int count = 0;
    struct Node *temp = head;

    while (temp != NULL) {
        count++;
```

```
        temp = temp->next;
    }

    return count;
}

// Display members
void display() {
    struct Node *temp = head;

    if (temp == NULL) {
        printf("Club is empty.\n");
        return;
    }

    printf("\nPRN\tName\n");

    while (temp != NULL) {
        printf("%d\t%s\n", temp->prn, temp->name);
        temp = temp->next;
    }
}
```

```
// Sort members by PRN
void sortList() {
    struct Node *i, *j;
    int prn;
    char name[50];

    for (i = head; i != NULL; i = i->next) {
        for (j = i->next; j != NULL; j = j->next) {
            if (i->prn > j->prn) {
                prn = i->prn;
                i->prn = j->prn;
                j->prn = prn;

                strcpy(name, i->name);
                strcpy(i->name, j->name);
                strcpy(j->name, name);
            }
        }
    }

    printf("List sorted by PRN.\n");
```

```
}

// Merge two lists
struct Node* mergeLists(struct Node *a, struct Node *b)
{
    if (a == NULL)
        return b;

    struct Node *temp = a;

    while (temp->next != NULL)
        temp = temp->next;

    temp->next = b;
    return a;
}

// Reverse list using pointers
void reverseList() {
    struct Node *prev = NULL;
    struct Node *curr = head;
    struct Node *next = NULL;
```

```
    while (curr != NULL) {
        next = curr->next;
        curr->next = prev;
        prev = curr;
        curr = next;
    }

    head = prev;
    printf("List reversed.\n");
}

int main() {
    int choice, prn;
    char name[50];
    struct Node *list2 = NULL;

    do {
        printf("\n--- PINNACLE CLUB ---\n");
        printf("1. Add President\n");
        printf("2. Add Member\n");
        printf("3. Add Secretary\n");
```

```
printf("4. Delete President\n");
printf("5. Delete Member\n");
printf("6. Delete Secretary\n");
printf("7. Count Members\n");
printf("8. Display Members\n");
printf("9. Sort List\n");
printf("10. Merge with Second List\n");
printf("11. Reverse List\n");
printf("0. Exit\n");
printf("Enter choice: ");
scanf("%d", &choice);

switch (choice) {
case 1:
    printf("Enter PRN and name: ");
    scanf("%d %49s", &prn, name);
    addPresident(prn, name);
    break;

case 2:
    printf("Enter PRN and name: ");
    scanf("%d %49s", &prn, name);
```

```
            addMember(prn, name);
            break;

        case 3:
            printf("Enter PRN and name: ");
            scanf("%d %49s", &prn, name);
            addSecretary(prn, name);
            break;

        case 4:
            deletePresident();
            break;

        case 5:
            printf("Enter PRN to delete: ");
            scanf("%d", &prn);
            deleteMember(prn);
            break;

        case 6:
            deleteSecretary();
            break;
```

```
case 7:
    printf("Total members: %d\n", countMembers());
    break;

case 8:
    display();
    break;

case 9:
    sortList();
    break;

case 10:
    printf("Enter second list member count: ");
    int n;
    scanf("%d", &n);

    for (int k = 0; k < n; k++) {
        printf("Enter PRN and name: ");
        scanf("%d %49s", &prn, name);
        addSecretary(prn, name);
```

```
            }

            list2 = NULL;
            printf("Second list members added to the club
list.\n");
            break;

        case 11:
            reverseList();
            break;

        case 0:
            printf("Exiting program.\n");
            break;

        default:
            printf("Invalid choice.\n");
        }
    } while (choice != 0);

    return 0;
}
```

Output :

--- PINNACLE CLUB ---

1. Add President

2. Add Member

3. Add Secretary

4. Delete President

5. Delete Member

6. Delete Secretary

7. Count Members

8. Display Members

9. Sort List

10. Merge with Second List

11. Reverse List

0. Exit

Enter choice: 1

Enter PRN and name: 101 Rahul

--- PINNACLE CLUB ---

Enter choice: 2

Enter PRN and name: 103 Amit

--- PINNACLE CLUB ---

Enter choice: 2

Enter PRN and name: 102 Priya

--- PINNACLE CLUB ---

Enter choice: 3

Enter PRN and name: 104 Sneha

--- PINNACLE CLUB ---

Enter choice: 8

PRN Name

101 Rahul

103 Amit

102 Priya

104 Sneha

--- PINNACLE CLUB ---

Enter choice: 7

Total members: 4

--- PINNACLE CLUB ---

```
Enter choice: 9
List sorted by PRN.

--- PINNACLE CLUB ---
Enter choice: 8

PRN     Name
101     Rahul
102     Priya
103     Amit
104     Sneha

--- PINNACLE CLUB ---
Enter choice: 5
Enter PRN to delete: 102
Member deleted.

--- PINNACLE CLUB ---
Enter choice: 8

PRN     Name
101     Rahul
```

```
103     Amit
104     Sneha

--- PINNACLE CLUB ---
Enter choice: 11
List reversed.

--- PINNACLE CLUB ---
Enter choice: 8

PRN     Name
104     Sneha
103     Amit
101     Rahul

--- PINNACLE CLUB ---
Enter choice: 4

--- PINNACLE CLUB ---
Enter choice: 8

PRN     Name
```

```
103     Amit
101     Rahul

--- PINNACLE CLUB ---
Enter choice: 6

--- PINNACLE CLUB ---
Enter choice: 8

PRN     Name
103     Amit

--- PINNACLE CLUB ---
Enter choice: 7
Total members: 1

--- PINNACLE CLUB ---
Enter choice: 0
Exiting program.
```